

Authentication

Lecture 7: Firebase Auth, OAuth providers, App Check

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro

Microsoft Teams course channel

rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with



TODAY

What you should leave with



AuthN vs AuthZ

Who the user is versus what they may do, and which layer answers each



Firebase Auth in Dart

Sign-up, sign-in, reset, verification, sign-out, and `authStateChanges()`



OAuth and tokens

The client-side credential handoff, and what an ID token proves



Rules and App Check

Rules that bind a request to a uid, and enforcement that keeps scripts out

PART 1 · FUNDAMENTALS

Who you are, and what you may do

Two different questions, answered by two different systems

Authentication is not authorization



Authentication

“Is this user really who they claim to be?”

It checks a credential (a password, a provider token, a passkey) and issues a session.



Authorization

“Now that I know who this is, what may they touch?”

It decides, per request, which documents this identity is allowed to read, create, update or delete.

Firestore Auth only answers the first question. The second is answered by your Security Rules and by your own backend. A signed-in user is not an authorized user, and confusing the two is a common Firestore mistake.

The three factors, and what MFA means

	What it means	Examples	How it fails
Knowledge	something you know	password, PIN, security answer	phished, reused, leaked in someone else's breach
Possession	something you have	phone for an OTP, passkey, hardware key	SMS is SIM-swappable; a passkey is not phishable
Inherence	something you are	fingerprint, Face ID, iris	unlocks a local secret; the biometric never leaves the device

Multi-factor means factors from different rows. A password plus a security question is one factor twice. Firebase supports SMS and TOTP second factors on the Identity Platform tier. Biometrics on the device are a screen lock, not a second factor.

Why you do not write this yourself

What Firebase gives you

- + Password hashing, salting and algorithm rotation you never touch
- + Verification, reset and OTP delivery, deliverable and rate-limited
- + 20+ federated providers behind one Dart API
- + Token minting, refresh and revocation, plus enumeration protection on by default

What you hand over

- Your identity records live in a Google project, not in your database
- Pricing tracks monthly active users once you leave the free tier
- Their email templates and flows: customizable, not arbitrary
- The uid becomes your primary key: export exists, migration is still work

Identity is a specialty, not a sprint task. Everything on the left is a place where a subtle bug becomes a breach. The right-hand column is the price, and for a semester project it is worth paying.

PART 2 · FIREBASE AUTH

firebase_auth, in Dart

Sign-up, sign-in, the auth gate, and the rest of the lifecycle

Providers, and the packages they need

	Provider	Flutter packages	Worth knowing
Native	Email & password	firebase_auth	enumeration protection is on by default
Native	Phone (SMS)	firebase_auth	costs money per message; SIM-swappable
Federated	Google	firebase_auth + google_sign_in	the flow we implement today
Federated	Apple	firebase_auth + sign_in_with_apple	required on iOS alongside other social logins
Anonymous	Anonymous	firebase_auth	a real uid before sign-up; link it later

One user, one uid, however many providers. Firebase keys the account on the email by default, so signing in with Google after signing up with that address lands on the same user.

Creating an account

```
final auth = FirebaseAuth.instance;

Future<void> signUp(String email, String password) async {
  try {
    final cred = await auth.createUserWithEmailAndPassword(
      email: email.trim(), password: password);
    await cred.user!.sendEmailVerification();
  } on FirebaseAuthException catch (e) {
    throw AuthFailure(switch (e.code) {
      'email-already-in-use' => 'That email is already registered.',
      'weak-password' => 'Use at least 8 characters.',
      'invalid-email' => 'Enter a valid email address.',
      _ => 'Sign-up could not be completed.',
    });
  }
}
```

Three things to notice

`createUserWithEmailAndPassword` signs the new user in immediately. The password never reaches your code again after this line returns.

Send the verification mail here, not later. An unverified account should not reach your data.

Sign-up is the one endpoint that still reveals existence: `email-already-in-use` is unavoidable on a usable form. Rate limiting and App Check are the mitigation, not a vaguer message.

Signing in, and the one message you show

```
Future<void> signIn(String email, String password) async {  
  try {  
    await auth.signInWithEmailAndPassword(  
      email: email.trim(), password: password);  
  } on FirebaseAuthException catch (e) {  
    // One message for every credential failure.  
    const generic = 'Email or password is incorrect.';  
    throw AuthFailure(switch (e.code) {  
      'too-many-requests' => 'Too many attempts. Try again later.',  
      'network-request-failed' => 'You appear to be offline.',  
      // invalid-credential, invalid-email, user-disabled, ...  
      _ => generic,  
    });  
  }  
}
```

Why one message

Since late 2023 Firebase no longer tells you which half was wrong: unknown email and wrong password both arrive as invalid-credential.

That is deliberate. Your job is to keep it that way in the UI.

Only two codes deserve their own message, and neither of them says anything about whether the account exists.

Never map user-disabled to its own message either: it confirms the account exists.

Account enumeration through the login form

“No user found for that email” turns the login form into a lookup service. Send it ten thousand addresses and it reports which ones have accounts, with no password and no breach involved.

The resulting list is itself sensitive data. On a dating app, a health app or a debt-management app, membership alone is the sensitive fact. It is also the first step of a credential-stuffing run: confirm which accounts exist, then attack only those addresses.

	The old deck taught	What you get today
Unknown email	user-not-found	invalid-credential
Wrong password	wrong-password	invalid-credential
Reset for an unknown email	user-not-found	succeeds silently
fetchSignInMethodsForEmail	the list of providers	an empty list, and deprecated

Identical message and identical behavior, whatever went wrong. Back it with rate limiting and App Check, and check the protection is still on in Console → Authentication → Settings.

authStateChanges() is the source of truth

```
// Don't check-and-push. Listen, and let the UI follow.
StreamBuilder<User?>(
  stream: FirebaseAuth.instance.authStateChanges(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const SplashScreen(); // session still restoring
    }
    final user = snapshot.data;
    if (user == null) return const LoginScreen();
    if (!user.emailVerified) return const VerifyEmailScreen();
    return const HomeScreen();
  },
)
```

Three streams, not one

`authStateChanges()`: sign-in, sign-out, and once at startup when the SDK restores the session from disk.

`idTokenChanges()`: all of that, plus every hourly token refresh and every custom-claim change.

`userChanges()`: plus profile edits such as `displayName`.

Never cache `currentUser` in a field of your own. It goes stale, and stale auth state is a security bug.

Route guards belong to the router, not to this widget. With `go_router` you feed this stream into `refreshListenable` and decide in `redirect` (→ Lecture 8).

Reset, verification, sign-out

```
// Password reset: never reveal whether the address exists.
await auth.sendPasswordResetEmail(email: email.trim());
showMessage('If that email has an account, a link is on its way.');
```



```
// Email verification: send it, then re-read after they tap.
await auth.currentUser?.sendEmailVerification();
await auth.currentUser?.reload();
final verified = auth.currentUser?.emailVerified ?? false;
```



```
// Sign-out: authStateChanges() then emits null.
await auth.signOut();
await GoogleSignIn.instance.signOut(); // end the provider session
```

The parts people get wrong

`emailVerified` on the cached `User` object does not update by itself. Call `reload()`, or listen to the stream, after the user follows the link.

`signOut()` clears the local session only. It does not revoke the refresh token server-side; that is an Admin SDK call.

Sign out of the federated provider too, or the next tap silently signs the same account straight back in.

Verification is a gate, not a notification. Enforce it in the auth gate and again in your security rules, with `request.auth.token.email_verified`.

Linking, re-authentication, second factors



Account linking

One person, one uid. `linkWithCredential` attaches Google to an existing email account, or upgrades an anonymous user without losing a single document.



Re-authentication

Changing an email or password, or deleting an account, needs a recent login. Catch `requires-recent-login` and call `reauthenticateWithCredential`.



Second factors

SMS and TOTP enrollment via `multiFactor`, on the Identity Platform tier. `local_auth`'s Face ID is a device lock on a session you already hold. It is useful, but it is not a factor.

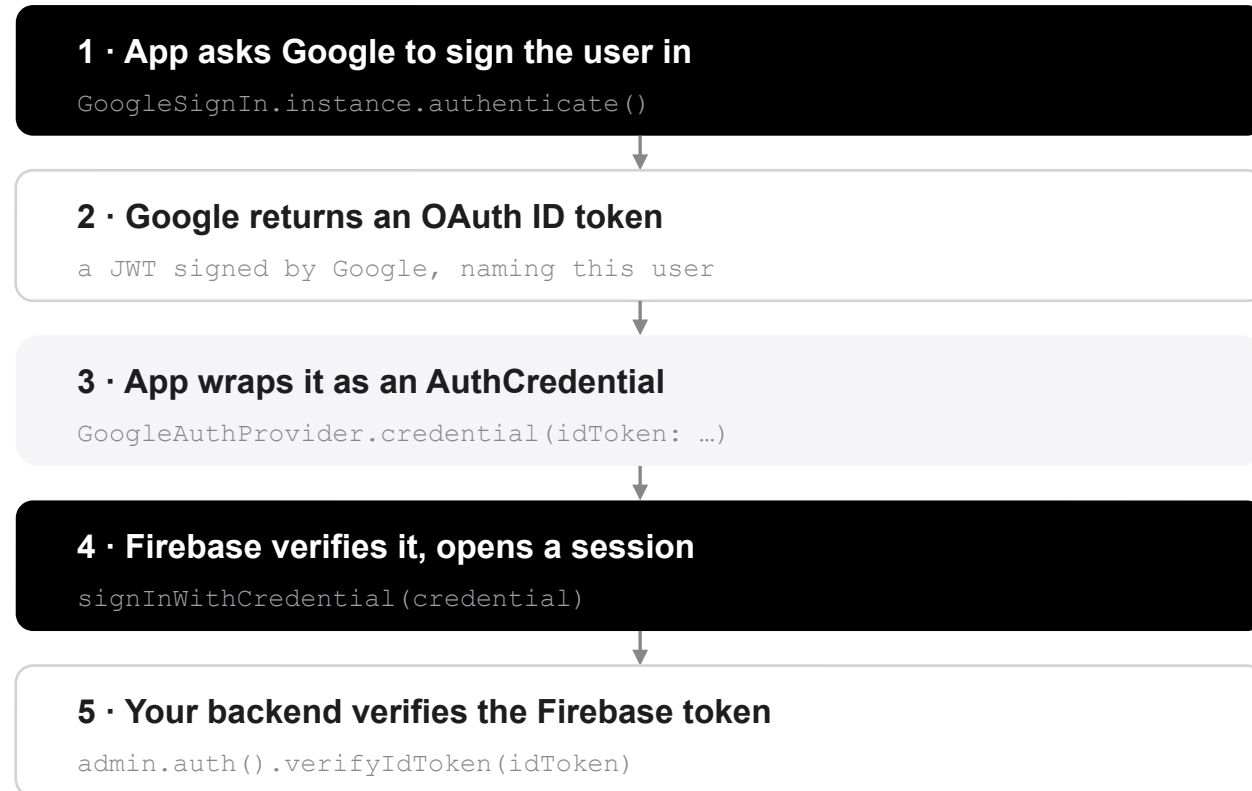
```
// Anonymous → real account: same uid, same data, nothing migrated.  
final cred = GoogleAuthProvider.credential(idToken: idToken);  
await FirebaseAuth.instance.currentUser!.linkWithCredential(cred);  
// on 'credential-already-in-use' the identity is taken: sign in with it
```

PART 3 · OAUTH & TOKENS

Borrowing an identity you did not verify

The credential handoff, and what the resulting token proves

The credential handoff, client-side



Every step runs on the device

This is the flow a Flutter app uses. Nothing here needs a server: the redirect-and-Cloud-Function variant is a different flow, for the web.

Firebase never sees the user's Google password. It checks Google's signature on the token, then mints a session of its own.

Same shape for Apple, Microsoft, GitHub and Facebook: obtain the provider credential, then hand it to `signInWithCredential`.

Step 5 is the one people skip. Your own API still has to verify the token it receives.

Google sign-in, end to end

```
import 'package:google_sign_in/google_sign_in.dart';

// google_sign_in 7.x: initialize once, at app start.
await GoogleSignIn.instance.initialize();

Future<UserCredential> signInWithGoogle() async {
  // 1. provider sign-in: the system account picker
  final account = await GoogleSignIn.instance.authenticate();
  // 2. Google's ID token for this user
  final idToken = account.authentication.idToken;
  // 3. wrap it as a Firebase credential
  final credential = GoogleAuthProvider.credential(idToken: idToken);
  // 4. exchange it for a Firebase session
  return FirebaseAuth.instance.signInWithCredential(credential);
}
```

Before it works

Android release builds need the SHA-1 and SHA-256 fingerprints registered in the Firebase console, or sign-in fails silently.

A canceled picker throws `GoogleSignInException` with code canceled. That is a user action, not an error to report.

Apple: `sign_in_with_apple`, then `OAuthProvider('apple.com').credential` with the `idToken` and the `rawNonce` you generated.

Offer Sign in with Apple on iOS whenever you offer any other social login; App Review enforces it.

ID token vs refresh token

	ID token	Refresh token
Lifetime	one hour, then the SDK refreshes it silently	until revoked, or the password changes
Format	a signed JWT: header.payload.signature	an opaque string, meaningful only to Google
Lives	in memory, handed out by getIdToken()	platform secure storage: Keychain, Keystore
Goes to your API	yes, as Authorization: Bearer <token>	never; it must not leave the device
Carries	uid, email, sign-in time, custom claims	nothing you can read

A JWT is signed, not encrypted. Anyone holding it can read the claims, so put nothing secret in a custom claim and treat a leaked ID token as a one-hour session hijack. `getIdToken(true)` forces a refresh, which you need after setting claims.

TOKENS

Verify on the backend; never trust a client uid

Client · Flutter

```
// Send the token. Never send the uid: it proves nothing.
final token = await FirebaseAuth.instance.currentUser!.getIdToken();
await http.post(url,
  headers: {'Authorization': 'Bearer $token'},
  body: jsonEncode({'item': itemId}));
```

Server · Node with firebase-admin

```
// Checks the signature, the expiry and the audience.
const token = req.get('Authorization').split(' ')[1];
const decoded = await admin.auth().verifyIdToken(token, true);
const uid = decoded.uid;    // trusted: Google signed this
await db.collection('orders').add({ uid, item: req.body.item });
```

Anyone can POST {"uid": "someone-else"}. The uid you act on must come out of a token you verified, in Firestore rules or in the Admin SDK, and from nowhere else. (→ Lecture 5, backends)

PART 4 · DATA, RULES & APP CHECK

Guarding the data, and the app itself

Where profiles live, rules that actually bind, and keeping scripts out

Never store the password

The identity provider holds the credential. Firebase keeps a salted script hash on Google's infrastructure; after `createUserWithEmailAndPassword` returns, the plaintext is gone from your process and never comes back.

Your database stores an identifier and profile data, and nothing that could authenticate anybody: no password, no PIN, no “security answer”, no copy of a token.

A password in Firestore is readable by everyone on your team in the console, and it ships to every client that one wrong rule lets through. Because users reuse passwords, one misconfiguration also exposes their accounts elsewhere.

The Firebase User object is for identity: uid, email, displayName, photoURL. Application data goes in your own collection, keyed by the uid.

```
// After sign-up: a profile doc keyed by the uid.  
final uid = cred.user!.uid;  
await FirebaseFirestore.instance  
    .doc('users/$uid')  
    .set({  
        'displayName': name,  
        'createdAt': FieldValue.serverTimestamp(),  
    });  
// No password. No token. No PIN.
```

One document per uid.

The uid is the join key between the identity system and your data, which is what the next slide has to defend.

Security rules that actually bind

```
// firestore.rules
match /users/{userId} {
  function isOwner() {
    return request.auth != null && request.auth.uid == userId;
  }
  allow read:    if isOwner();
  allow create:  if isOwner()
                  && !request.resource.data.keys().hasAny(['role']);
  allow update:  if isOwner()
                  && request.resource.data.diff(resource.data)
                      .affectedKeys().hasOnly(['displayName']);
  allow delete:  if false;    // deletion runs on the backend
}
```

Rules run on Google's servers, so they cannot be bypassed. That is why they must be written as if every client is hostile.

A rule that does not bind

```
allow create: if request.auth != null;
```

Any signed-in user, including an account created seconds ago, can write a document at someone else's uid and take over their profile.

`request.auth != null` is authentication. Authorization is binding the request to the path it is touching.

Split the verbs. create, update and delete need different conditions, and a blanket write hides all three.

Test in the console's Rules Playground, and in CI against the emulator suite (→ Lecture 5).

The other question: is this request from my app?

Your Firebase configuration (API key, project id, app id) ships inside the binary. It identifies the project. It was never a secret, and it never protected anything.

An attacker unzips the APK, reads it, and calls the Firestore or Storage REST API from a Python script, so none of the limits your UI imposes apply.

The cost is yours: millions of reads, a poisoned collection, a scraped user table. Auth does not help here, because the script can sign up like any other user.

Auth answers who. App Check answers where from: is this request coming from a genuine build of my app, on a genuine device?

App Check

“Is this my real, unmodified app?”

protects the developer: abuse, billing fraud

Firestore Auth

“Is this a known, valid user?”

protects the user: unauthorized access

Security Rules

“May this user do this, here?”

protects the data: misuse by valid users

Three layers, three different questions.

Turning it on: activate() and providers

```
import 'package:firebase_app_check/firebase_app_check.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);
  // Activate BEFORE the first call to any Firebase service.
  await FirebaseAppCheck.instance.activate(
    androidProvider: kDebugMode
      ? AndroidProvider.debug : AndroidProvider.playIntegrity,
    appleProvider: kDebugMode
      ? AppleProvider.debug : AppleProvider.appAttest,
  );
  runApp(const App());
}
```

Pick the right attestor

Android

Play Integrity: the app was installed by Play, unmodified

iOS 14+

App Attest: a hardware-backed key per install

older iOS

DeviceCheck: device-level, weaker but broad

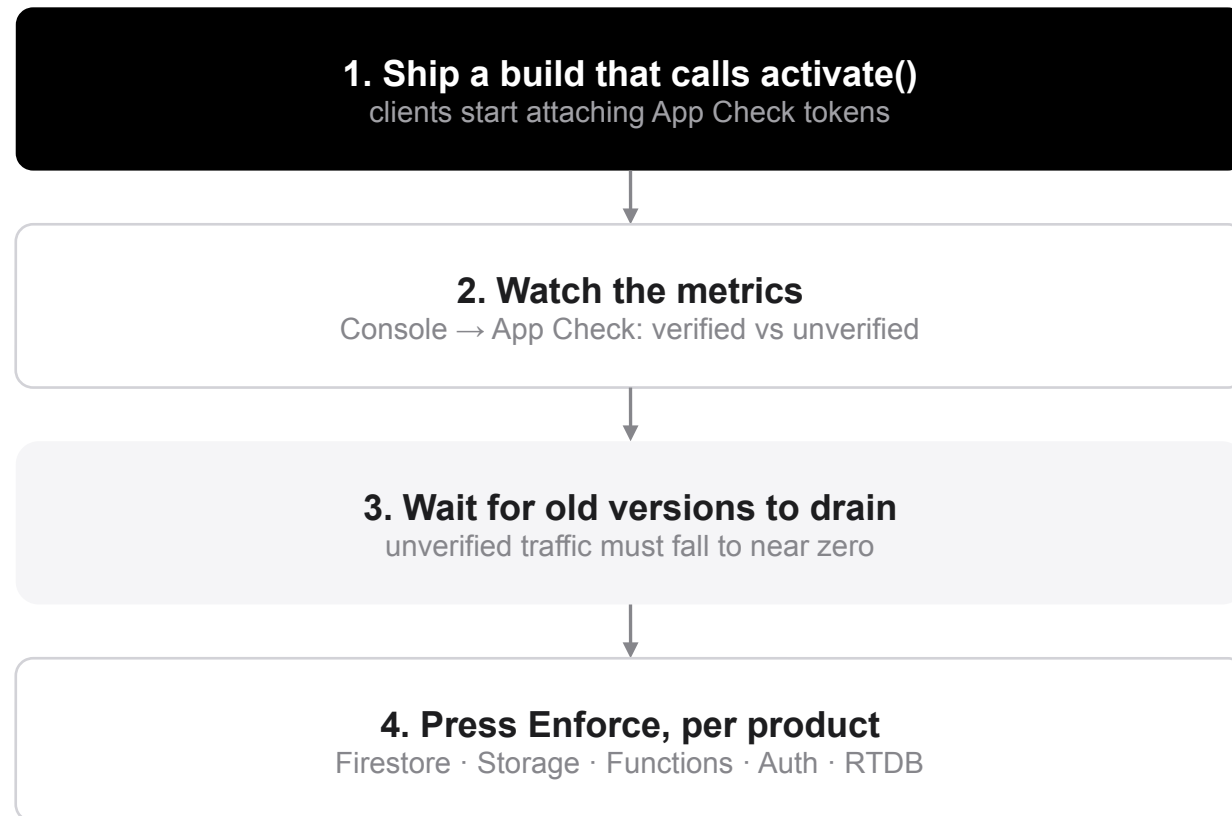
Web

reCAPTCHA Enterprise

Dev & CI

Debug provider: emulators and simulators cannot attest

Enforcement: the step that changes behavior



Debug tokens, safely

Run with the debug provider; the console prints a token on first launch.

Register it in Console → App Check → Manage debug tokens, give it a name, set an expiry.

A debug token is a permanent bypass of every enforcement you just enabled. Never commit one, never paste one in chat, rotate them when someone leaves the team.

On CI, inject it as a secret, never as a checked-in file.

Until you press Enforce, App Check only measures.

The SDK call on its own changes nothing; that is where the old version of this lecture stopped.

Before next week



Recap

AuthN is who, AuthZ is what: Firebase Auth and Security Rules are different layers

One generic message for every failed sign-in; never confirm an account exists

`authStateChanges()` is the source of truth; never cache `currentUser`

Verify the ID token on the backend; a uid from the client proves nothing



This week

Add email/password and Google sign-in to your project app

Write the `users/{userId}` rules and break them in the Rules Playground

Wire up reset, verification and sign-out

Turn on App Check with the debug provider and read the metrics page



Read more

firebase.google.com/docs/auth/flutter/start

firebase.google.com/docs/rules/rules-and-auth

firebase.google.com/docs/app-check

OWASP MASVS · Authentication and Session Management